

1. Structural graph analysis (Leiden vs k-core on 40k nodes)

On a 40K-node, 43K-edge sparse wiki graph, high-performance libraries are crucial. For example, **graph-tool** (C++ with Python bindings) is extremely fast: in published tests on ~40K nodes it computed k-core in 0.0033 s using 16 threads (vs igraph's ~0.0098 s and NetworkX's ~0.72 s) ¹. **igraph** (C) is also very fast (single-thread k-core ~0.01 s) while **NetworkX** (pure Python) is orders of magnitude slower (~0.7 s) ¹. On GPUs, NVIDIA's **cuGraph** dramatically accelerates community detection: on much larger graphs (3.8M nodes) Leiden ran ~8–48× faster than CPU libs ². At our 40K scale, we'd still expect cuGraph Leiden to finish in sub-second on a GPU, while CPU libraries (igraph, graph-tool) take milliseconds. PyTorch Geometric's **PyFlagser** (Flagser C++ backend) can compute homology (Betti numbers) of the graph very quickly via its optimized C++ core ³ ⁴.

Memory: Storing the sparse 40K×40K adjacency or Laplacian is trivial (<100MB). NetworkX's overhead is high (our trial used ~160 MB), whereas igraph/graph-tool use far less. PyFlagser and scipy's sparse use minimal memory for this size.

Complementarity of k-core and Leiden: Modularity-based community detection like Leiden can be unstable on very sparse graphs (it tends to overfit noise or split weakly connected components ⁵ ⁶). In practice, one can first prune the graph by taking a high-*k*-core (iteratively removing low-degree nodes) to extract the “well-connected backbone” ¹. This k-core backbone then yields more meaningful Leiden clusters. Thus we'd run a small *k* core (e.g. *k*=2 or 3) to focus on the dense core, and apply Leiden there. We would verify by subsampling or synthetic tests: e.g. apply modularity clustering on random sparse graphs (where ground truth is empty) vs on their k-cores, to check stability.

Libraries: igraph (Python port) and NetworkX are easy to use but NetworkX is very slow ¹. graph-tool (C++) is blazing fast but heavy to install (needs compilation). cuGraph (if GPU available) offers massive speedups ². PyFlagser (scikit-tda) provides C++ performance for homology ³. All cited repos are active (e.g. graph-tool, igraph), except pure Python NetworkX (very old) and Leidenalg (igraph extension) which has some recent commits.

Verification: We would validate on a sample graph of ~10K nodes by timing each library's Leiden and k-core, checking memory use (e.g. using Linux tools or memory_profiler). For scalability, we'd examine graph-tool's CPU use (multithreading) and cuGraph's GPU memory. We'd ensure that a 10× growth in edges (i.e. ~430K edges) still runs in seconds. A useful metric is how often tiny degree nodes (<2) appear: if >50% of nodes are degree 0–1, Leiden may produce trivial communities, so k-core filtering is needed. If problems arise (e.g. Leiden crashes or returns one community), we'd fallback to k-core+Leiden or label propagation.

2. Graph Transformers vs GNNs vs Sheaf NNs on sparse heterophilic graph

For a sparse, heterophilic knowledge graph, simpler architectures may be preferable. Many graph transformer models (e.g. **GraphGPS** ⁷ ⁸, **Graphormer** ⁹, **NAGphormer** ¹⁰) shine on large *homophilic* benchmarks (e.g. molecular graphs), but their overhead (multi-headed self-attention over many nodes) is heavy. For example, GraphGPS has 848 GitHub stars but its code was last updated Feb 2023 ⁷ (stale), whereas NAGphormer (135★) is active into 2024 ¹¹. By contrast, **GraphSAGE** and **GAT** are light-weight and scale to millions of nodes (e.g. PinSage for product graphs), but they tend to struggle when neighbors have different labels.

Recent “heterophily-aware” approaches include **Neural Sheaf Diffusion (NSD, 2022)** and **SGPC (2025)**, which explicitly model higher-order topology. NSD (Twitter-Research repo, 92★) showed competitive results on small heterophilic benchmarks (Cornell, Texas, Wikipedia pages) ¹². SGPC (arXiv 2508.00357) reports *state-of-the-art* on 9 benchmarks (mix of homophilic/heterophilic) using a PAC-Bayes spectral loss and a Wasserstein-based sheaf lifting ¹³. It even provides confidence bounds on consistency. Thus SGPC’s complexity may pay off: it unifies optimal-transport lifts and variance-reduced diffusion ¹³. However, SGPC is very new (0★ repo, Nov 2025) and not battle-tested. NSD’s repo has one contributor and 2022 code ¹².

In summary: **If heterophily is severe**, sheaf-based models offer benefits (NSD/SGPC), at the cost of complex setup. **Graph transformers** like Graphormer (2.4k★, updated 2024 ⁹) or GraphGPS (848★) have no published heterophily results, so they may not justify their cost here. We lean toward a sheaf GNN: perhaps NSD or SGPC. SGPC’s OT-lift seems promising, but it requires training a global optimizer (not federated-friendly). Bosca-Ghrist’s “local discrepancy” training ¹⁴ avoids backprop, which could align well with a multi-wiki setup.

Repositories: We note GraphGPS ⁷ and Graphormer ⁹ codebases (some example models exist but focus on molecular tasks), NAGphormer ⁸ code is active. NSD code (twitter-research/neural-sheaf-diffusion) ¹⁵ and SGPC code ¹⁶ are newer; their dependencies (PyTorch, SciPy) are maintained.

Verification: We would benchmark on a representative heterophilic task (e.g. node classification on a similar text-based wiki graph). If none exist, one could create labels (e.g. flags for node roles). We’d compare Graphormer/GPS (via PyG+CUDA) vs GraphSAGE vs NSD/SGPC in terms of accuracy and train time. If the complex models show <5% gain, simpler GNNs win. We’d also examine embeddings: do sheaf methods yield qualitatively different clusters? 8 Verify by probing for over-smoothing (diffusion causing feature collapse) or oversensitivity (attention spreading noise). If SGPC’s training is unstable at 40K nodes, we’d fall back to NSD or even plain GAT.

3. Sheaf Laplacian and Fiedler eigenvector at 40K nodes

Computing a **sheaf Laplacian** on 40K nodes is feasible but requires care. In the simplest case (each node stalk is 1D), the Laplacian is a 40k×40k sparse matrix, which we can build with SciPy. Finding the Fiedler

(second-smallest) eigenvector via `scipy.sparse.linalg.eigsh` is memory- and time-intensive but doable (10–60 s, ~100–200 MB). If each node has higher-dimensional stalks (e.g. 5D claims), the matrix grows to 200k×200k, making it harder (might need ~1 GB, and more sophisticated solvers like ARPACK with preconditioning). However, the latest sheaf-based theory (Bosca–Ghrist 2026) suggests constructing the Laplacian only on “free” coordinates, which are usually limited.

Libraries: We can use **SciPy** (sparse) for linear algebra, which is up-to-date. **PySheaf** (Python) lets us define cellular sheaves ¹⁷, but it does not include a large-scale eigen-solver. We’d export to SciPy from `pysheaf` (last commit Nov 2024 ¹⁸, Python 3.x). For topology: **Dionysus 2** (C++) ¹⁹ and **Ripser.py** (C++ core) ²⁰ can compute H_0/H_1 of the graph quickly (these are stable, actively maintained). They can verify, e.g., that H_0 equals number of components, $H_1 \sim E - V + \text{components}$. **PyFlagser** (Giotto AI) also computes directed homology. All have modern wheels (no forks needed).

Bosca–Ghrist bottleneck: Their “Fiedler-eigenvector bottleneck” method (arXiv:2603.14831) likely requires constructing a modified Laplacian or gradient flow. We’d have to implement their algorithm atop SciPy or PyTorch. If it needs repeated eigen solves, it might become a bottleneck (hour-scale) on 40K nodes. We’d prototype on a 5K-node sample first to measure memory/time.

Verification: We’d write a test sheaf (e.g. identity restriction plus random small sheaf constraints) on a 10K-node subgraph. We’d time `eigsh(L, k=2)` and check $\lambda_2 > 0$ (connectivity). We’d also run PyFlagser’s homology to check Betti numbers. If ARPACK fails to converge or is very slow, we’d try Trilinos/Anasazi or a PyTorch sparse eigensolver. Any naive full-matrix method is impossible at this scale. In summary, nothing should “break” given modern sparse solvers: the main risk is slow convergence if the graph is nearly disconnected ($\lambda_2 \approx 0$). We’d handle that by adding small diagonal noise or using shift-invert.

4. Learnable restriction maps (cross-wiki alignment)

The idea is to learn mappings between entity spaces of different wikis (P2P, Wikidata, etc.) via patterns of co-links and chunk embeddings. This is essentially a KG alignment task. The **SGPC (Wasserstein Entropic Lift)** approach would treat each wiki canon as a discrete distribution and find an optimal-transport “lift” between them, learning restriction maps via central optimization ¹³. In contrast, **Bosca–Ghrist’s local discrepancy updates** train maps by purely local consistency rules (no global backprop) ¹⁴, which is more federation-friendly (no shared gradients).

No known production system does exactly this. In practice, multi-wiki alignment is done via record linkage: e.g. using **OpenRefine’s Wikidata reconciliation** or **Mix’n’Match** for semi-automatic linking of external items to Wikidata. The **Wikidata Integrator** (Python library) automates writing reconciled entities ²¹. But these rely on string matching or manual curation, not on embedding patterns. Academic work on KG alignment often uses embeddings (e.g. Faiss or gemini-like OT matches), but not in a live wiki federation. A survey of multi-ontology alignment found no service at the 5–20K entity scale aside from project-specific efforts.

Recommendation: We would experiment with both. SGPC’s OT-lift could be applied by treating co-occurrence of wikilinks as training pairs, and using chunk embeddings to refine. Bosca–Ghrist’s local

updates could iteratively adjust the maps based on disagreement at edges (their Fig. on local discrepancy ¹⁴). Before deployment, we'd test on known alignments (e.g. seed a set of common entities like "Michel Bauwens") to see if the model recovers correct mappings. We'd check metrics like precision/recall against a held-out alignment. If SGPC's full solve is too heavy, local updates offer a fallback. If neither reaches >70% precision on sample alignments, we'd revert to manual/heuristic linking.

5. Poset cohomology on MediaWiki categories

Treat the **MediaWiki category graph** as a poset (DAG of parent→child categories) with claims as sheaf values on nodes. Computing H^0 would find globally consistent sections (e.g. a global assignment of truth values to claims), while H^1 would highlight "obstructions" (contradictions across cycles of categories). This is novel: to our knowledge no one has computed sheaf cohomology on wiki categories.

Pipelines: In principle, **pysheaf** can represent a sheaf on a cell complex ¹⁷. We could model each category as a 0-cell, each subcategory relation as a 1-cell, and define restriction maps based on how claims propagate. Then using **pysheaf** we could compute cochains and their (co)boundary. However, **pysheaf** does not have built-in cohomology solvers; we'd have to set up matrices and call SciPy to solve for kernels.

Dionysus, **Gudhi**, or **Ripser** focus on persistent homology of simplicial data, not on sheaf cohomology. We could use **Dionysus** to compute classical homology of the category graph (ignoring claims), but that isn't the same.

Libraries: None of **Dionysus**/**Gudhi**/**Ripser** provide sheaf cohomology out-of-the-box. **Ripser.py** is focused on point-cloud VR complexes ²². **Dionysus 2** is maintained (156★) ²³ and could compute Betti-0/1 of an abstract complex if we encode the categories as simplices. But pulling in claims as stalks is beyond its scope. The sheaf theory here would largely be custom.

Novelty: Since the only reference we see is theory (e.g. arXiv 2502.15476 extends cohomology to Morse posets), this application appears novel. There are no production examples of sheaf H^1 on knowledge graphs. We'd likely need to prototype a simple case: e.g. a small category graph of 100 nodes, with trivial sheaf (identity maps) to validate $H^0 = \#$ connected components, $H^1 = 0$. Then add artificial disagreements and see if H^1 detects them. Key libraries: **pysheaf** (active), **SciPy** (for linear algebra). If **pysheaf** proves too limited, a custom Python implementation of Čech/cohomology on the DAG may be needed.

6. Discourse extraction (Q, Claim, Evidence, Support/Contradict)

Argument mining in theoretical text is challenging. State-of-art uses large LLMs with prompting and retrieval. Recent surveys note that techniques like chain-of-thought prompting and retrieval-augmented generation have greatly improved AM tasks ²⁴. For P2P prose, we would try a multi-stage strategy: (a) **Few-shot prompting with examples**. Provide examples of P2P paragraphs labeled with (question, claim, evidence) triples to GPT-4 or Claude. CoT (explain reasoning) might boost precision. (b) **Retrieval**: use a RAG approach where relevant domain documents (or wiki pages) are retrieved to ground the model's answers. This often improves factuality and context adherence. (c) **Fine-tuning**: if possible, we'd fine-tune a

moderately sized LLM (e.g. LLaMA-2 or Flan-T5) on annotated P2P argument data. The **SIDE** framework (Meta 2022) or specialized models (Wanda, NeoWiki) might also be adapted here, but none are known to directly output (Q,C,E) structures.

To reach ≥ 0.8 F1 on a gold P2P set, we expect that vanilla prompting might achieve ~ 0.6 – 0.7 (on known datasets LLMs often need coaxing) ²⁴. RAG could add 5–10 points by injecting factual context. Fine-tuning a smaller model on ~ 1000 examples from DiscourseGraphs or Araucaria might push us past 0.8.

Verification: We'd create a gold dev set of P2P theoretical passages manually labeled. Then compare strategies: prompt-only vs prompt+RAG vs fine-tuned model. A prior analysis suggests that retrieval+CoT often outperforms plain few-shot ²⁴. We'd measure precision/recall of extracted (Q,C,E) spans. If any approach falls short, we'd increase examples or consider chain-of-thought and self-consistency voting (asking model to list reasons) as in AM literature.

7. Dynamic/streaming updates (10–50 edits/day)

With only ~ 10 – 50 edits per day, updates are trivial for most frameworks. The key is incremental maintenance. For communities: libraries like puzzlef's **Louvain-communities (OpenMP)** support dynamic updates (though code is a few years old), and preliminary work on **Dynamic Leiden** exists in literature. In practice, simply re-running Leiden on 40K nodes (very sparse) takes < 1 s with igraph/graph-tool, so full recompute is cheap.

For embeddings: **InkStream** (2309.11071) implements incremental GNN inference on streaming graphs. It showed 2.4–343 $\times$ speedups over full re-run on large graphs ⁴, suggesting we could use a similar approach. Streaming GraphSAGE could be done via PyG Temporal or writing custom incremental steps.

For the database side: **Pathway** (Python) is designed for streaming ETL with a differential-dataflow engine that *incrementally* updates results ²⁵. It allows connecting to Postgres/pgvector (via connectors) and maintaining real-time analytics. Similarly, tools like **Feldera** offer SQL-like continuous queries over streams (the blog demonstrates real-time feature queries) and could be adapted to graph analytics ²⁶. Systems like **Materialize** and **RisingWave** support materialized views over Postgres/JSONB: e.g. RisingWave's Postgres sink ²⁷ means you could write a SQL transform and have it auto-maintained as the edits stream in.

Minimizing re-extraction: On an edit, only the changed node and its neighbors truly need reprocessing. Community labels can be updated locally if the change is small. For embeddings, one can do a single forward/backward sweep through affected edges (as InkStream does). In practice, with 50 edits/day, we might simply schedule hourly mini-batches.

Verification: We will simulate edit streams: e.g. apply 50 random edits (new pages or changed links), and measure the time to update communities (Leiden re-run vs incremental), update embeddings (full batch vs InkStream-like incremental), and refresh any materialized views. We expect all to be sub-second. We'll watch memory (should be constant, since working sets are small) and latency per edit (target < 100 ms with Pathway or similar). If any part exceeds our SLA (e.g. if an edit triggers a large chain, we'll log and tune).

- 1 performance – graph-tool: Efficient network analysis with Python
<https://graph-tool.skewed.de/performance.html>
- 2 How to Accelerate Community Detection in Python Using GPU-Powered Leiden | NVIDIA Technical Blog
<https://developer.nvidia.com/blog/how-to-accelerate-community-detection-in-python-using-gpu-powered-leiden/>
- 3 pyflagser/RELEASE.rst at master · giotto-ai/pyflagser · GitHub
<https://github.com/giotto-ai/pyflagser/blob/master/RELEASE.rst>
- 4 [2309.11071] InkStream: Real-time GNN Inference on Streaming Graphs via Incremental Update
<https://arxiv.org/abs/2309.11071>
- 5 6 Modularity maximization considered harmful – Inverse Complexity Lab
<https://skewed.de/lab/posts/modularity-harmful/>
- 7 GitHub - rampasek/GraphGPS: Recipe for a General, Powerful, Scalable Graph Transformer · GitHub
<https://github.com/rampasek/graphgps>
- 8 11 GitHub - JHL-HUST/NAGphormer: NAGphormer: A Tokenized Graph Transformer for Node Classification in Large Graphs · GitHub
<https://github.com/JHL-HUST/NAGphormer>
- 9 GitHub - microsoft/Graphormer: Graphormer is a general-purpose deep learning backbone for molecular modeling. · GitHub
<https://github.com/microsoft/graphormer>
- 10 [2206.04910] NAGphormer: A Tokenized Graph Transformer for Node Classification in Large Graphs
<https://arxiv.org/abs/2206.04910>
- 12 [2202.04579] Neural Sheaf Diffusion: A Topological Perspective on Heterophily and Oversmoothing in GNNs
<https://arxiv.org/abs/2202.04579>
- 13 [2508.00357] Sheaf Graph Neural Networks via PAC-Bayes Spectral Optimization
<https://arxiv.org/abs/2508.00357>
- 14 Top arXiv papers
<https://scirate.com/?date=2026-03-17&page=18&range=3>
- 15 GitHub - twitter-research/neural-sheaf-diffusion · GitHub
<https://github.com/twitter-research/neural-sheaf-diffusion>
- 16 GitHub - ChoiYoonHyuk/SGPC: AAAI 2026 · GitHub
<https://github.com/ChoiYoonHyuk/SGPC>
- 17 GitHub - kb1dds/pysheaf: Python Cellular Sheaf Library · GitHub
<https://github.com/kb1dds/pysheaf>
- 18 Commits · kb1dds/pysheaf · GitHub
<https://github.com/kb1dds/pysheaf/commits/master/>
- 19 23 GitHub - mrzv/dionysus: Library for computing persistent homology · GitHub
<https://github.com/mrzv/dionysus>
- 20 22 GitHub - scikit-tda/ripser.py: A Lean Persistent Homology Library for Python · GitHub
<https://github.com/scikit-tda/ripser.py>

21 GitHub - giotto-ai/pyflagser: Python bindings and API for the flagser C++ library (<https://github.com/luetge/flagser>). · GitHub

<https://github.com/giotto-ai/pyflagser>

24 Large Language Models in Argument Mining: A Survey

<https://arxiv.org/html/2506.16383v3>

25 GitHub - pathwaycom/pathway: Python ETL framework for stream processing, real-time analytics, LLM pipelines, and RAG. · GitHub

<https://github.com/pathwaycom/pathway>

26 Real-time feature engineering with Feldera. Part 2.

<https://www.feldera.com/blog/feature-engineering-part2>

27 Sink data from RisingWave to PostgreSQL

<https://docs.risingwave.com/integrations/destinations/postgresql>